

INF333 2023-2024 Spring Semester

Lone Wolf

Muhammet Kerim Yilmaz <muhammetkerim.yilmaz2@gsu.edu.tr>

Homework I Design Document

Please provide answers inline in a `quote` environment.

1 Preliminaries

Q1: If you have any preliminary comments on your submission, notes for the TAs, or extra credit, please give them here.

Q2: Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, and lecture notes.

- <https://web.stanford.edu/ouster/cgi-bin/cs140-spring20/pintos/pintos1.html>

2 Sleep

2.1 Data Structures

Q3: Copy here the declaration of each new or changed ‘struct’ or ‘struct’ member, global or static variable, ‘typedef’, or enumeration.

Identify the purpose of each in 25 words or less.

Answer here

2.2 Algorithms

Q4: Briefly describe your implementation of `thread_join()` and how it interacts with thread termination.

the thread_join function searches for the specified child thread in the parent's list of children. It then acquires the

Q5: What steps are taken to minimize the amount of time spent in the timer interrupt handler?

Answer here

2.3 Synchronization

Q6: Consider parent thread P with child thread C. How do you ensure proper synchronization and avoid race conditions when P calls `wait(C)` before C exits? After C exits? How do you ensure that all resources are freed in each case? How about when P terminates without waiting, before C exits? After C exits? Are there any special cases?

To ensure synchronization and prevent race conditions:

P calls `wait(C)` before C exits → P locks C, waits on a condition variable, and releases the lock when C signals its exit. P calls `wait(C)` after C exits → P directly cleans up C's resources after acquiring a lock. P terminates before `wait(C)`, while C is running → C becomes an orphan; the kernel handles cleanup after C exits. P terminates after C exits → If P hasn't waited, C's resources are freed when P exits.

Proper locking and cleanup prevent resource leaks and race conditions.

Q7: How are race conditions avoided when multiple threads call `timer_sleep()` simultaneously?

disabling interrupts during list operations. This ensures atomic updates, prevents interference, and maintains a consistent sleep list.

Q8: How are race conditions avoided when a timer interrupt occurs during a call to `timer_sleep()`?

Race conditions in `timer_sleep()` are avoided by disabling interrupts during list operations. This ensures that sleep

2.4 Rationale

Q9: Critique your design, pointing out advantages and disadvantages in your design choices.

Advantages:

Prevents Race Conditions: Disabling interrupts ensures that multiple threads don't interfere with each other. Good Synchronization: Using locks and condition variables avoids busy-waiting and improves CPU efficiency. Proper Resource Cleanup: The parent thread waits for the child and cleans up resources, preventing memory leaks.

Disadvantages:

Too Much Locking: Frequent locks might slow down performance when many threads are active. Priority Issues: Lower-priority threads might be delayed if high-priority threads keep running. Complex Cleanup: If *thread_join* is not handled well, some thread resources might not be freed properly.

3 Priority Scheduling

3.1 Data Structures

Q10: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less

- sleeping_list
- propagate_priority
- lock_acquire
- priority
- compare_locks
- compare_elements

Q11: Describe the sequence of events when a call to `lock_acquire()` causes a priority donation. How is nested donation handled?

When a thread waits for a lock in `lock_acquire()`, it checks if its priority is higher than the lock holder's effective priority. If so, the function `set_effective_priority()` updates the lock holder's priority to match the waiting thread's priority.

To handle nested donation, `set_effective_priority()` recursively updates priorities through the lock-thread chain. If the lock holder is also waiting for another lock, the priority update continues to ensure correct donation.

Q12: Describe the sequence of events when `lock_release()` is called on a lock that a higher-priority thread is waiting for.

- Remove the lock from the thread's held locks list.
- Recalculate effective priority as the maximum of the thread's original priority and remaining donations.
- Unblock the highest-priority waiting thread and grant it the lock.
- If the unblocked thread has higher priority, trigger a context switch.

3.2 Synchronization

Q13: Describe a potential race in `thread_set_priority()` and explain how your implementation avoids it. Can you use a lock to avoid this race?

- A thread setting its priority may override an active priority donation.
- Disabling interrupts prevents race conditions by ensuring atomic updates.
- Using locks could cause deadlocks due to interactions with priority donation.

3.3 Rationale

Q14: Why did you choose this design? In what ways is it superior to another design you considered?

- Maintaining sorted wait lists was complex and unnecessary.
- Selecting the highest-priority thread at scheduling time is simpler and faster.
- Sorting adds overhead ($O(n \log n)$), while linear selection ($O(n)$) is more efficient.

4 Advanced Scheduler

4.1 Data Structures

Q15: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less.

Answer here

4.2 Algorithms

Q16: Suppose threads A, B, and C have nice values 0, 1, and 2. Each has a recent_cpu value of 0. Fill in the table below showing the scheduling decision and the priority and recent_cpu values for each thread after each given number of timer ticks:

timer ticks	recent_cpu			priority			thread to run
	A	B	C	A	B	C	
0							
4							
8							
12							
16							
20							
24							
28							
32							
36							

Q17: Did any ambiguities in the scheduler specification make values in the table uncertain? If so, what rule did you use to resolve them? Does this match the behavior of your scheduler?

Answer here

Q18: How is the way you divided the cost of scheduling between code inside and outside interrupt context likely to affect performance?

Answer here

4.3 Rationale

Q19: Briefly critique your design, pointing out advantages and disadvantages in your design choices. If you were to have extra time to work on this part of the project, how might you choose to refine or improve your design?

Answer here

Q20: The assignment explains arithmetic for fixed-point math in detail, but it leaves it open to you to implement it. Why did you decide to implement it the way you did? If you created an abstraction layer for fixed-point math, that is, an abstract data type and/or a set of functions or macros to manipulate fixed-point numbers, why did you do so? If not, why not?

Answer here

5 Survey Questions

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want—these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

Q1: In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

Answer here

Q2: Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

Answer here

Q3: Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

Answer here

Q4: Do you have any suggestions for the TAs to more effectively assist students, either for future quarters or the remaining projects?

Answer here

Q5: Any other comments?

Answer here